

Nature-Inspired AI

Exam Preparation Worksheet (2025/26)

Miguel Cabral Pinto & amigos

Made with the context of the entire Nature-Inspired AI / Evolutionary Computation course from MEI @ UC. Questions marked ★ are taken verbatim (or near-verbatim) from the official Mock Exam. Questions marked ★ are adapted from a past compilation of written-exam questions by Prof. Ernesto Costa, restricted to topics that overlap with this year's syllabus. Questions marked ★ are adapted from past real written exams (2021–2022), covering topics not already addressed by the questions above.

Contents

Key Equations Reference	2
Algorithm Quick Reference	4
1 Introduction: Problem Solving & Heuristics	5
2 Classical Meta-Heuristics	6
3 Evolutionary Algorithms: The General Framework	7
4 Genetic Algorithms: Representations & Operators	9
5 Genetic Programming	11
6 Evolutionary Strategies	13
7 Differential Evolution	15
8 Collective Intelligence: Boids, PSO & ACO	16
9 Co-Evolutionary Algorithms	18
10 Multi-Objective Evolutionary Algorithms	20
11 Evolutionary Machine Learning	21
Answer Key	23

Key Equations Reference

Benchmark: Rastrigin function

$$f(\mathbf{x}) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)], \quad f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y), \quad x, y \in [-5.12, 5.12]$$

Global minimum at (0,0), $f(0,0) = 0$. Used throughout the course to visually compare Random Search, Hill Climbing, SA, ES and CMA-ES.

Simulated Annealing – acceptance criterion

$$P(\text{accept worse solution}) = \exp\left(-\frac{\Delta E}{T}\right), \quad \Delta E = f(\text{new}) - f(\text{current})$$

High $T \Rightarrow$ exploration (worse solutions often accepted); low $T \Rightarrow$ exploitation (greedy).

Genetic Algorithms – real-valued crossover

$$\text{Arithmetical: } x' = \alpha x_1 + (1 - \alpha)x_2, \quad \alpha \in [0, 1] \quad (1)$$

$$\text{Heuristic: } x' = \alpha(x_2 - x_1) + x_2, \quad \alpha \in [0, 1], \quad x_2 \text{ the better parent} \quad (2)$$

$$\text{BLX-}\alpha: x' \sim U[\min(x_1, x_2) - \alpha\Delta, \max(x_1, x_2) + \alpha\Delta], \quad \Delta = |x_2 - x_1| \quad (3)$$

Particle Swarm Optimization

$$v_{\text{inertial}}^{(j)} = k_{\text{inertial}} \times v^{(j-1)} \quad (4)$$

$$v_{\text{cognitive}}^{(j)} = k_{\text{cognitive}} \times (x_{\text{best}} - x^{(j-1)}) \quad (5)$$

$$v_{\text{social}}^{(j)} = k_{\text{social}} \times (x_{\text{global-best}} - x^{(j-1)}) \quad (6)$$

$$v^{(j)} = v_{\text{inertial}}^{(j)} + v_{\text{cognitive}}^{(j)} + v_{\text{social}}^{(j)}, \quad x^{(j)} = x^{(j-1)} + v^{(j)} \quad (7)$$

$$\chi = \frac{2}{|2 - c - \sqrt{c^2 - 4c}|}, \quad c = k_{\text{cognitive}} + k_{\text{social}} > 4 \quad (\text{constriction coefficient}) \quad (8)$$

Ant Colony Optimization

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \times \eta_{ij}^\beta}{\sum_{h \in J_k} \tau_{ih}^\alpha \times \eta_{ih}^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}} \quad (9)$$

$$\tau_{ij}(t+1) = (1 - \rho) \tau_{ij}(t) + \Delta \tau_{ij}(t), \quad \Delta \tau_{ij}^k(t) = \begin{cases} Q/L_k(t) & \text{if } (i, j) \in T_k(t) \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Differential Evolution

$$v_i = x_1 + \gamma(x_2 - x_3) \quad (\text{DE/rand/1, mutant vector}) \quad (11)$$

$$u_{ik} = \begin{cases} v_{ik} & \text{if } (r < r_{\text{cross}}) \text{ or } (i = i_{\text{rand}}) \\ x_{ik} & \text{otherwise} \end{cases} \quad (\text{binomial crossover}) \quad (12)$$

Notation DE/x/y/z: $\mathbf{x}$ = base vector type, $\mathbf{y}$ = # difference vectors, $\mathbf{z}$ = crossover type.

Evolutionary Strategies notation

(1+1)-ES: 1 parent, 1 offspring, elitist. (μ, λ) -ES: μ parents, λ offspring, **non-elitist** (next gen drawn only from offspring). $(\mu+\lambda)$ -ES: parents and offspring compete together, **elitist**.

CMA-ES (one generation, using N_{best} selected individuals)

$$\mu_x^{(g+1)} = \frac{1}{N_{\text{best}}} \sum_{i=1}^{N_{\text{best}}} x_i, \quad \sigma_x^{2,(g+1)} = \frac{1}{N_{\text{best}}} \sum_{i=1}^{N_{\text{best}}} (x_i - \mu_x^{(g)})^2 \quad (\text{note: uses the } \textit{old} \text{ mean } \mu^{(g)})$$

Natural Evolution Strategies

$$\nabla_{\theta} \mathbb{E}[f(x)] \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(x_i) \cdot f(x_i), \quad \theta_{t+1} = \theta_t + \eta \cdot \text{FIM}^{-1} \cdot \nabla_{\theta} \mathbb{E}[f(x)]$$

Multi-objective optimization

Pareto dominance (minimization): $x \preceq y \iff \forall i : x_i \leq y_i \wedge \exists k : x_k < y_k$. NSGA-II crowding contribution: $\frac{f_{\text{next}} - f_{\text{prev}}}{f_{\text{max}} - f_{\text{min}}}$ (summed over all objectives; boundary solutions get ∞).

NEAT

$$\delta = c_1 \frac{E}{N} + c_2 \frac{D}{N} + c_3 \bar{W}, \quad f'(i) = \frac{f(i)}{\sum_{j=1}^n s(\delta(i, j))}$$

E = excess genes, D = disjoint genes, $\bar{W}$ = avg. weight diff. of matching genes, N = size of larger genome.

Algorithm Quick Reference

	Representation	Main variation	Selection	Typical domain
GA	Binary / real / permutation vector	Crossover (1-pt, uniform, order, PMX) + mutation	Roulette wheel / tournament	Combinatorial & continuous opt. (knapsack, scheduling)
ES	Real vector + strategy params (σ)	Gaussian mutation (crossover minor/absent)	Deterministic, (μ, λ) or $(\mu+\lambda)$	Continuous parameter optimisation
DE	Real vector	Differential mutation + binomial crossover	Deterministic greedy (trial vs. target)	Continuous, multimodal numeric optimisation
GP	Tree (or linear / graph)	Subtree crossover (90%) / subtree mutation (10%)	Fitness-proportional parents, generational survivors	Symbolic regression, program synthesis
PSO	Position + velocity (real)	Velocity update (inertia + cognitive + social)	Implicit (personal-best / global-best memory)	Continuous optimisation, swarm robotics
ACO	Pheromone matrix over graph	Stochastic construction (pheromone + heuristic)	Implicit (pheromone deposit/evaporation)	Combinatorial graph problems (TSP, routing)

1 Introduction: Problem Solving & Heuristics

Before any algorithm, we must classify the problem (design, learning, or optimization) and the kind of solver available (analytical, numerical, or observational). Many real problems (TSP, n -body) have no efficient exact solver, motivating heuristics: a representation of the search space, a search strategy to explore it, and a quality measure to evaluate candidates. This triad reappears, refined, in every algorithm of the course.

1.1 Classify each scenario as a design, learning, or optimization problem: (a) tuning the weights of a neural network to fit data; (b) laying out the floor plan of a new building; (c) minimizing $\|Dw - d\|^2$ over w .

1.2 Distinguish analytical, numerical, and observational solvers. For the Travelling Salesman Problem, explain why an analytical solver is impractical and a numerical (heuristic) solver is preferred.

1.3 A heuristic's general approach has three components. Name them and explain, for the knapsack problem, what each one would concretely be.

1.4 Evaluate the following claim: "Since heuristics do not guarantee an optimal solution, they are inferior to exact algorithms and should only be used when no exact algorithm exists."

1.5 The n -body problem and the Travelling Salesman Problem are both given as motivating examples in the introduction. What do they have in common that makes them suitable examples for a course on heuristics, despite belonging to very different problem domains (continuous physics vs. discrete combinatorics)?

1.6 ★ The No Free Lunch (NFL) Theorem states that, averaged over all possible problems, no algorithm performs better than random search. A student concludes: "Therefore, comparing algorithms with statistical tests is pointless, and designing specialized heuristics is a waste of effort." Comment on this reasoning.

2 Classical Meta-Heuristics

Meta-heuristics are general-purpose, stochastic search strategies that trade optimality guarantees for tractability on hard problems (non-differentiable, non-convex, combinatorial, constrained). This section covers the three foundational examples: Random Search (pure stochastic exploration, no learning), Hill Climbing (greedy local search, several variants, prone to local optima/plateaus), and Simulated Annealing (probabilistic acceptance of worse moves, controlled by a temperature schedule, to escape local optima).

2.1 Define “meta-heuristic” and list its three key characteristics (general-purpose, stochastic, exploration vs. exploitation).

2.2 List the four reasons given for why classical (exact/gradient-based) optimization methods are insufficient for many real-world problems.

2.3 Random Search and Hill Climbing both explore a search space, but differ fundamentally in how they use feedback. Explain this difference, and state which one provides a meaningful “baseline” for comparison and why.

2.4 Which of the following best describes Steepest-Ascent Hill Climbing?

- A) Evaluate neighbors one by one in random order and move to the first improving one.
- B) Evaluate all neighbors, then move to the one with the best improvement.
- C) Choose a neighbor probabilistically, weighted by its degree of improvement.
- D) Accept any neighbor, better or worse, with fixed probability 0.5.

2.5 List and briefly explain three weaknesses of Hill Climbing (e.g. local optima, plateaus, lack of exploration).

2.6 Explain how Simulated Annealing’s temperature parameter controls the exploration / exploitation trade-off, and why a probabilistic acceptance of worse solutions helps escape local optima where Hill Climbing would get stuck.

2.7 Using the acceptance criterion $P = \exp(-\Delta E/T)$, compute (approximately) the acceptance probability of a move with $\Delta E = 2$ at $T = 1$ and at $T = 10$. What does the comparison tell you about high vs. low temperature?

2.8 For the 2-D Rastrigin function $f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y)$, compute $f(0, 0)$ and $f(1, -1)$. Given these two values, explain why a Hill-Climbing algorithm started near $(1, -1)$ could fail to ever reach $(0, 0)$.

2.9 Compare linear, exponential, and logarithmic cooling schedules for Simulated Annealing in terms of how quickly they reduce T , and the practical trade-off each implies between solution quality and runtime.

2.10 True or false: “Given the same starting point, standard (non-stochastic) Hill Climbing always produces the same final solution.” Justify.

3 Evolutionary Algorithms: The General Framework

Evolutionary Algorithms (EAs) generalize the meta-heuristic idea into a population-based, fitness-driven, stochastic, iterative search. Every EA instantiates the same four components: representation, fitness evaluation, selection, and variation. This section uses the Knapsack Problem as a running example to illustrate binary representation, constraint handling, parent selection (roulette wheel, tournament), survivor selection (elitism), and variation (flip mutation, one-point crossover).

3.1 Define an Evolutionary Algorithm and list its four key characteristics.

3.2 Why do EAs not require the objective function to be differentiable or convex? Relate your answer to the limitations of gradient-based methods discussed in Section 2.

3.3 Name and briefly explain the four key components of the general EA pipeline, using the Knapsack Problem to illustrate each.

3.4 Compare Roulette Wheel Selection and Tournament Selection as parent-selection mechanisms. What determines an individual's chance of being selected in each?

3.5 In the Knapsack example, an invalid solution (total weight $>$ capacity) is simply assigned $\text{Fit}(x) = 0$. Discuss one drawback of this constraint-handling strategy compared to a penalty proportional to the constraint violation.

3.6 Explain elitism as a survivor-selection mechanism and state one risk of using it too aggressively (e.g. with a very large elite size).

3.7 ★ A Genetic Algorithm population contains four individuals (A, B, C, D) with fitness values (maximization) $\text{Fit}(A) = 10$, $\text{Fit}(B) = 20$, $\text{Fit}(C) = 5$, $\text{Fit}(D) = 15$. Using standard Roulette Wheel Selection, what is the probability that individual B is selected exactly twice in two independent selection spins?

- A) 0.16 (or 16%)
- B) 0.04 (or 4%)
- C) 0.40 (or 40%)
- D) 0.25 (or 25%)

3.8 ★ You run a GA with Tournament Selection and observe premature convergence to local optima. To reduce selection pressure and encourage exploration, what should you do to the tournament size k ?

- A) Increase k , because a larger tournament increases the probability that lower-fitness individuals are selected.
- B) Set $k = 1$, which maximizes selection pressure by enforcing strict elitism.
- C) Switch to a dynamic k that scales proportionally with the maximum fitness of the population.
- D) Decrease k , because a smaller tournament reduces the probability that the absolute best individuals dominate the selection pool.

3.9 List two challenges of EAs in general (e.g. computational expense, parameter sensitivity) and one mitigation for each.

3.10 ★ Comment on the following claim: “In an evolutionary algorithm, parent selection should depend only on the individual’s fitness.”

3.11 ★ Comment on the following claim: “Keeping the mutation and crossover probabilities fixed throughout the run is detrimental to the evolutionary process.”

3.12 ★ Roulette Wheel Selection assumes nonnegative fitness values. If your fitness function can output negative numbers, can the roulette wheel principle still be adapted to this situation? Explain how (or why not).

3.13 ★ Survivor selection is an important design choice in every EA. Describe and compare the following mechanisms: (a) full generational replacement, (b) steady-state replacement, (c) elitism, (d) age-based replacement. For each, state its main advantage and one drawback in terms of diversity and convergence.

4 Genetic Algorithms: Representations & Operators

Genetic Algorithms (Holland, 1970s) instantiate the EA framework with three classic representations — binary, real-valued, and permutation — each with its own dedicated mutation and crossover operators. Binary uses flip mutation and one-/two-point/uniform crossover; real-valued uses uniform/Gaussian mutation and arithmetical/heuristic/blend (BLX- α) crossover; permutations use swap/insert/scramble mutation and order/PMX/cycle crossover to preserve validity.

4.1 For each representation (binary, real-valued, permutation), give one problem type it is naturally suited for and explain why.

4.2 Describe one-point, two-point, and uniform crossover for binary strings. Which tends to disrupt building blocks the most, and why?

4.3 Describe flip mutation for a binary representation, using a 6-bit example.

4.4 For real-valued representations, contrast mutation by a uniform distribution with mutation by a Gaussian distribution. Which gives finer control over step size, and why?

4.5 Given the arithmetical crossover formula $x' = \alpha x_1 + (1 - \alpha)x_2$, compute x' for $x_1 = 2$, $x_2 = 10$, $\alpha = 0.25$. What is the geometric interpretation of α ?

4.6 Heuristic crossover is defined as $x' = \alpha(x_2 - x_1) + x_2$ where x_2 is the fitter parent. Explain why this operator is described as “directional” and how it differs in spirit from arithmetical crossover.

4.7 For a permutation representation, describe Swap, Insert, and Scramble mutation, and explain why a naive flip-style mutation (as used for binary strings) cannot be applied directly.

4.8 Order Crossover (OX), Partially Mapped Crossover (PMX), and Cycle Crossover are all designed to solve the same underlying problem when recombining two permutations. What problem is that, and why does naive one-point crossover fail for permutations?

4.9 ★ A student switches a GA from a binary string (with standard binary-to-gray code mapping) representation to a continuous real-valued vector representation for a high-precision engineering problem, using optimal variation operators for both. What is a fundamental structural advantage of the real-valued representation in this context?

- A) It prevents the phenomenon of genetic drift across the population.
- B) It eliminates the problem of ‘Hamming cliffs’, where adjacent phenotypic values require multiple simultaneous bit flips.
- C) It completely removes the need for mutation operators, relying solely on directional crossover.
- D) It guarantees that the schema theorem holds strictly, which is mathematically impossible with binary representations.

4.10 ★ Two parents in a real-valued GA have single-variable chromosome values $x_1 = 2.0$ and $x_2 = 6.0$. Applying Blend Crossover (BLX- α) with $\alpha = 0.5$, which range represents the complete interval from which an offspring’s value can be uniformly sampled?

- A) [1.0, 7.0]
- B) [-2.0, 10.0]
- C) [2.0, 6.0]
- D) [0.0, 8.0]

4.11 Walk through Partially Mapped Crossover (PMX) step by step on two small permutations of your choice (length 5), showing the mapping section and the repair of duplicate genes.

4.12 ★ Suppose you apply standard one-point crossover to two parent chromosomes of length n , where every gene differs between the two parents (i.e. $x_A[i] \neq x_B[i]$ for all i). Assuming the cut point is chosen from the $n - 1$ internal positions (so some non-empty segment is always exchanged), in what situations, if any, would an offspring be identical to one of the parents? What is the probability of this happening?

4.13 ★ Design a Genetic Algorithm to solve Sudoku (fill a 9×9 grid, respecting the given clue cells, so that every row, column, and 3×3 block contains the digits 1–9 exactly once). Specify, and justify, only: the representation, the fitness function, the crossover operator, and the mutation operator.

4.14 ★ Comment on the following claim: “The existence of a representation gap between genotype and phenotype is always a drawback for evolutionary algorithms, since it adds unnecessary complexity to the mapping.”

4.15 ★ The Boolean Satisfiability Problem (SAT) asks: given a propositional formula in conjunctive normal form (e.g. $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_2 \vee \overline{x_4}) \wedge (x_1 \vee \overline{x_3} \vee x_5)$ with 5 variables), find a truth assignment that makes the formula evaluate to True. Design a nature-inspired solution to SAT, justifying your choice of representation, fitness function, and variation operators.

5 Genetic Programming

Genetic Programming (Koza, 1992) evolves programs represented as trees built from a function set and a terminal set, instead of fixed-length vectors. Variation is either crossover (subtree exchange, ~90%) or mutation (subtree replacement, ~10%); populations are seeded via Full, Grow, or Ramped Half-and-Half initialization. GP faces unique challenges — closure, introns, bloat, and overfitting — and has several non-tree variants: Grammatical Evolution (genotype = integer codons mapped through a grammar), Linear GP (sequential register instructions), and Cartesian GP (graph encoded as an integer list).

5.1 Define the function set and terminal set of a tree-based GP individual. Give one example of each for a symbolic-regression problem over real numbers.

5.2 In standard Koza-style GP, crossover and mutation are described as “mutually exclusive” with typical probabilities of 90% and 10%. Explain what “mutually exclusive” means here and describe subtree crossover and subtree mutation.

5.3 Compare the Full, Grow, and Ramped Half-and-Half population initialization methods. Why is Ramped Half-and-Half generally preferred?

5.4 ★ In standard, untyped Tree-based Genetic Programming (Koza-style), the property of closure is a strict requirement. Which of the following best describes what closure enforces?

- A) Every tree in the population must evaluate to a boolean value at its root node to allow selection to occur.
- B) The mutation operator must only replace a subtree with an identical syntactic structure to preserve tree depth.
- C) The terminal set must contain only constants, ensuring that execution terminates within a finite number of steps.
- D) Any function in the function set must be able to accept any value returned by any other function or terminal as an argument.

5.5 GP trees often contain “introns” — inviable or unoptimized code that does not affect the output. Explain why introns can be *beneficial* despite contributing nothing to fitness (hint: how destructive can mutation be in trees?).

5.6 Define bloat. Give two of the proposed causal theories (introns, drift, crossover bias) and two countermeasures (e.g. size penalty, double tournament, depth limits).

5.7 Why is GP particularly prone to overfitting, and what two mitigations does the course suggest (Occam’s razor / simplicity preference, validation set)?

5.8 For the Maze Navigation example, $\text{fitness} = (\text{units_spent} + \text{distance_to_cheese})^2$, with a maximum of 100 units. Compute the fitness of a solution that spends 60 units and ends 5 units from the cheese. Then explain why the formula squares the sum rather than using it directly: what effect does squaring have on the penalty for large versus small combined errors, and why might this shape be desirable for guiding GP search?

5.9 Describe the Grammatical Evolution genotype-to-phenotype mapping pipeline: what is the genotype, and how is each codon used to select a production rule from the grammar?

5.10 ★ In Grammatical Evolution, the genotype is a variable-length string of integer codons, mapped to a phenotype via a Context-Free Grammar. A non-terminal node currently being mapped has 3 production rules available. If the current codon value is 26, which production rule is selected?

- A) The 3rd rule (Index 2)
- B) The 2nd rule (Index 1)
- C) The 1st rule (Index 0)
- D) The mapping fails, triggering structural wrapping immediately.

5.11 Describe Linear GP's representation (instruction sequence operating on registers) and how its variation operators (mutation, two-point crossover) work.

5.12 ★ In Linear Genetic Programming (LGP), programs are sequences of instructions operating on registers. What structural characteristic explicitly distinguishes LGP from tree-based GP regarding instruction execution?

- A) LGP variations must map every register back to a separate leaf node in a Context-Free Grammar prior to fitness evaluation.
- B) LGP contains structural introns—instructions whose outputs do not affect the final program output, meaning segments of the genotype can be entirely neutral.
- C) LGP completely eliminates the possibility of infinite loops because all programs must follow strict tree-depth limits.
- D) LGP requires all instructions to return boolean values to ensure proper evaluation execution.

5.13 Cartesian GP encodes a program as a list of integers representing a directed acyclic graph. Describe what the function genes and connection genes represent, and why the connectivity constraints (involving the levels-back parameter l) are needed.

5.14 Using the function lookup table $0=+$, $1=-$, $2=*$, $3=/$ and the genotype 0 0 1 1 0 0 1 3 1, manually trace the first two nodes built by Cartesian GP (inputs are nodes 0 and 1).

5.15 ★ John Koza, the inventor of Genetic Programming, claimed that the mutation operator is not necessary in GP, and that crossover alone is sufficient for effective search. Comment on this claim: why did Koza argue this, and is the argument convincing?

5.16 ★ Define *protected division* as a mechanism for achieving closure in GP. Why does including standard (unprotected) division in the function set violate closure, and how does protected division fix this?

5.17 ★ Comment on the following claim: “Tree-based Genetic Programming has two major drawbacks as a representation: high redundancy and low locality.” Define each property, and evaluate whether the claim is accurate.

6 Evolutionary Strategies

Evolutionary Strategies (Rechenberg & Schwefel, 1960s) emphasize mutation (Gaussian noise) and selection over crossover. The simple notations $(1+1)$, (μ, λ) , and $(\mu+\lambda)$ differ in elitism. CMA-ES adapts the mutation step size and direction via a covariance matrix estimated from the best individuals each generation; NES instead optimizes the parameters of a sampling distribution directly via the natural gradient, using information from all individuals (not just the best).

6.1 Define Evolutionary Strategies and contrast their emphasis (mutation + selection) with the Genetic Algorithm's emphasis on crossover.

6.2 Explain the $(1+1)$ -ES procedure. Why is it described as elitist?

6.3 ★ What is a critical operational difference between a (μ, λ) Evolutionary Strategy and a $(\mu + \lambda)$ Evolutionary Strategy?

- A) In (μ, λ) , the parents are completely discarded, and the next generation's μ parents are selected only from the λ new offspring.
- B) (μ, λ) requires that $\mu > \lambda$, whereas $(\mu + \lambda)$ requires that $\lambda > \mu$.
- C) In (μ, λ) , the mutation rate is automatically adapted, whereas in $(\mu + \lambda)$ it remains static.
- D) (μ, λ) guarantees elitism, while $(\mu + \lambda)$ allows the average fitness of the population to decrease over generations.

6.4 Explain why (μ, λ) -ES, despite being non-elitist and risking the loss of good solutions, can be advantageous for maintaining diversity compared to $(\mu + \lambda)$ -ES.

6.5 What shortcoming of simple ES (fixed mutation step size σ) motivates CMA-ES?

6.6 In CMA-ES, the covariance matrix $C^{(g+1)}$ for the next generation is computed using the best N_{best} individuals of generation g , but using the *old* mean $\mu^{(g)}$ rather than the newly computed $\mu^{(g+1)}$. Why might the algorithm be designed this way rather than using $\mu^{(g+1)}$?

6.7 You have two best-selected 1-D points $x_1 = 4, x_2 = 8$ and the (old) mean $\mu_x^{(g)} = 5$. Using $\sigma_x^{2,(g+1)} = \frac{1}{N_{\text{best}}} \sum (x_i - \mu_x^{(g)})^2$ with $N_{\text{best}} = 2$, compute $\sigma_x^{2,(g+1)}$. Then interpret the result: what does a large σ^2 imply about the spread of the next generation's samples, and what phase of the search (exploration vs. exploitation) does this correspond to?

6.8 NES's motivation is that "bad solutions are important too." Explain what information CMA-ES and simple ES discard that NES retains, and how NES uses it (optimizing distribution parameters via natural gradient).

6.9 Explain the role of the Fisher Information Matrix (FIM) in the NES update rule, and why the natural gradient is preferred over the standard Euclidean gradient for this purpose.

6.10 Sketch the full CMA-ES generational loop (the four-step procedure: evaluate fitness, isolate N_{best} , compute new covariance using the old mean, sample new candidates from $\mathcal{N}(\mu^{(g+1)}, C^{(g+1)})$).

6.11 ★ Define *self-adaptation* as it is used in Evolution Strategies. How does it differ from a fixed mutation step size σ , and why is self-adaptation generally preferable?

7 Differential Evolution

Differential Evolution represents candidates as real vectors and creates variation geometrically: a mutant vector is built from the weighted difference of two randomly chosen individuals added to a third (the base vector), then mixed with the target via binomial crossover, and the better of target/trial survives deterministically. The $DE/\mathbf{x}/\mathbf{y}/\mathbf{z}$ notation summarizes the base-vector strategy, the number of difference vectors, and the crossover type.

7.1 List the four steps of the DE algorithm (initialization, mutation, crossover, selection) and briefly describe each.

7.2 Given the mutant vector formula $v_i = x_1 + \gamma(x_2 - x_3)$, explain the role of the scale factor γ and what happens (qualitatively) if γ is very large vs. very small.

7.3 Binomial crossover guarantees that the trial vector u_i differs from the target x_i in at least one gene, via a special index i_{rand} . Explain why this guarantee is necessary (what would happen without it?).

7.4 Describe DE's survivor selection rule. Is it stochastic or deterministic, and how does this compare to Simulated Annealing's acceptance rule?

7.5 Decode the notation $DE/\text{best}/2/\text{bin}$: what does each component mean, and what is the expected effect of using **best** as the base vector and 2 difference vectors instead of **rand**/1?

7.6 Given $x_1 = [1.0, 2.0]$, $x_2 = [4.0, 6.0]$, $x_3 = [2.0, 3.0]$ and $\gamma = 0.5$, compute the mutant vector $v_i = x_1 + \gamma(x_2 - x_3)$. Then explain geometrically what the vector $(x_2 - x_3)$ represents, and why using population members to define a search direction makes DE fundamentally different from Gaussian mutation.

7.7 Compare the random scale factor schedule $\gamma = 0.5(1 + U(0, 1))$ with the linear schedule $\gamma = \gamma_{\text{max}} - (\gamma_{\text{max}} - \gamma_{\text{min}}) \frac{\text{iter}_{\text{cur}}}{\text{iter}_{\text{max}}}$ in terms of how each balances exploration and exploitation over a run.

8 Collective Intelligence: Boids, PSO & ACO

Collective Intelligence emerges from autonomous agents with no centralized control, interacting locally according to simple rules (Boids). Particle Swarm Optimization adapts this to numerical optimization: particles move according to inertia, a cognitive term (pull toward their own best), and a social term (pull toward the swarm's best), with mechanisms (clamping, decreasing inertia, constriction, predator-prey) to balance exploration/exploitation and diversity. Ant Colony Optimization instead uses indirect, chemical (stigmergic) communication: ants probabilistically choose paths based on pheromone and heuristic information, then deposit and evaporate pheromone to reinforce good solutions over time.

8.1 List the three defining characteristics of the Collective Intelligence model (autonomous agents, no centralized control, neighborhood/topology + rules of behavior).

8.2 Explain, in your own words, what “the whole is greater than the sum of its parts” means in the context of Boids: how can simple local rules between individual agents produce complex global flocking behavior?

8.3 PSO’s velocity update is motivated by an analogy: a particle is simultaneously “brave” (keeps flying its own direction), “conservative” (returns to its own best), and a “swarm member” (moves toward the swarm’s best). Map each behavior to its corresponding velocity term (inertial, cognitive, social).

8.4 ★ In standard PSO, a particle updates its velocity using inertia, a cognitive component, and a social component. If you set both the cognitive acceleration coefficient (c_1) and the social acceleration coefficient (c_2) to exactly 0, how will the particles behave?

- A) The particles will immediately freeze in place at their current spatial coordinates.
- B) The particles will continue moving in straight lines along their current velocity vectors, completely ignoring their own past success and the success of the swarm.
- C) The particles will instantly collapse into the geometric center of the initial swarm.
- D) The particles will perform a pure random walk, as the velocity vector becomes entirely determined by stochastic noise.

8.5 ★ A particle in a 2D PSO space resides at position $X_t = [4.0, -2.0]$ with instantaneous velocity $V_t = [1.0, 3.0]$. The swarm determines the velocity update for the next time step to be $V_{t+1} = [-2.0, 1.5]$. What is the particle’s updated position X_{t+1} ?

- A) $[2.0, 1.0]$
- B) $[3.0, 4.5]$
- C) $[5.0, 1.0]$
- D) $[2.0, -0.5]$

8.6 Explain velocity clamping ($[-v_{\max}, v_{\max}]$) and why it is needed to balance exploration and exploitation.

8.7 Describe the decreasing-inertia schedule for k_{inertial} across generations. Why does promoting exploration early and exploitation later make sense for PSO?

8.8 How does the constriction coefficient χ achieve a similar goal to velocity clamping (stability/convergence) through a different mechanism?

8.9 Define the swarm activity diversity metric $sa = \frac{1}{n} \sum_i |x_i(t) - x_i(t-1)|$, and explain how Predator-Prey PSO uses a repulsion term to maintain diversity.

8.10 Explain how Binary PSO converts a continuous velocity into a binary decision using the sigmoid function $f(x) = 1/(1 + e^{-x})$.

8.11 Explain stigmergy and why it is classified as “indirect” communication, contrasting it with the “direct” physical (antennation) and non-physical (stridulation) communication used by other social insects.

8.12 Outline the Ant System algorithm: how does an ant build a solution step by step, and what happens to pheromones at the end of each iteration?

8.13 ★ An ant at node i must choose between path A and path B, with $\alpha = 1$ (pheromone importance) and $\beta = 2$ (heuristic importance). Path A: $\tau_A = 2$, $\eta_A = 3$. Path B: $\tau_B = 4$, $\eta_B = 1$. What is the exact probability p_A that the ant selects Path A?

- A) 0.818
- B) 0.500
- C) 0.600
- D) 0.182

8.14 ★ In Ant Colony Optimization, if the pheromone evaporation rate parameter (ρ) is set too high (i.e. approaching 1), what failure mode is most likely to occur in the swarm’s behavior?

- A) The colony will fail to build up structural memory of good paths, causing the ants to behave almost randomly and lose exploitation capability.
- B) The swarm will experience stagnation, where all ants permanently follow the exact same sub-optimal trail.
- C) The heuristic information (η) will completely override the pheromone matrix, turning the algorithm into a deterministic greedy search.
- D) The convergence rate will accelerate exponentially, finding the global optimum within the first few iterations.

8.15 In the combined pheromone+heuristic transition rule $p_{ij}^k \propto \tau_{ij}^\alpha \eta_{ij}^\beta$, explain what happens to ant behavior in the limits $\alpha \gg \beta$ and $\alpha \ll \beta$.

8.16 Contrast the standard Ant System pheromone update (all ants deposit) with the Elitist Ant System (best-so-far solution gets extra deposit) and Rank-Based Ant System (deposit weighted by rank). What problem does each successive variant try to address?

8.17 Explain the “random keys” encoding that allows PSO (a continuous-space algorithm) to be applied to permutation problems.

8.18 Besides Predator-Prey PSO, two other mechanisms for keeping diversity were mentioned: Random Immigrants, and a Forgetting mechanism. Explain what each one does.

9 Co-Evolutionary Algorithms

Co-evolution generalizes the EA fitness function to depend on the interaction between individuals from multiple populations, $f : X_1 \times \dots \times X_n \rightarrow E$, rather than evaluating individuals in isolation. Competitive co-evolution (adversarial, e.g. games) pits individuals against each other and risks pathologies like cycling, disengagement, overspecialization, forgetting, and the Red Queen Effect. Cooperative co-evolution (e.g. classifier rules) decomposes a problem across populations and must solve credit assignment: who deserves credit for a good joint solution?

9.1 Write the formal definition of fitness for a standard EA, $f : X \rightarrow E$, and contrast it with the co-evolutionary fitness function $f : X_1 \times \dots \times X_n \rightarrow E$. What does this change imply about evaluating individuals in isolation?

9.2 Explain the difference between the “solution concept” typical of competitive co-evolution (an element of X_i) and that typical of cooperative co-evolution (a combination of elements across the X_i ’s).

9.3 Give one example each of a competitive and a cooperative co-evolutionary setup (individual representation and fitness signal for each).

9.4 Distinguish subjective fitness (measured against the current population) from objective fitness (measured against all possible opponents). Why is subjective fitness used in practice despite being a less reliable measure of true quality?

9.5 In two-population competitive co-evolution, candidate solutions S and tests T are kept in separate populations. Explain why separating “solving” from “providing search gradient” can be advantageous compared to one-population co-evolution.

9.6 Explain three of the following pathologies of one-population competitive co-evolution: cycling, disengagement, overspecialization (focusing), forgetting.

9.7 ★ In a competitive co-evolutionary system (e.g. a predator-prey or attacker-defender configuration), algorithms often face the challenge of the Red Queen Effect. Which option precisely encapsulates why standard fitness metrics break down under this effect?

- A) The fitness landscape is dynamic and relative; as individuals in population A adapt to outperform population B, population B also shifts, meaning absolute generational fitness improvements can appear flat or non-existent.
- B) The population sizes of both species oscillate according to strict Lotka-Volterra differential equations, leading to mandatory premature convergence.
- C) Both populations quickly discover structural introns that block any variation operators from altering phenotypic outcomes.
- D) The selection pressure drops to zero because individual successes are evaluated entirely on independent, fixed objective test benches.

9.8 Cooperative co-evolution is motivated by a “divide and conquer” rationale. List two benefits this offers (e.g. modularity, smaller projected search spaces) and explain the central difficulty it introduces (credit assignment).

- 9.9** List two open questions in cooperative co-evolution discussed in the slides (e.g. how to build a team, credit to team vs. members, diversity maintenance, specialization vs. generalization).
- 9.10** For a one-population competitive co-evolution with $n = 10$ individuals, compute the number of pairwise interactions required by (a) round-robin, and (b) a single-elimination tournament.
- 9.11** Sketch a fitness function design for the Co-evolutionary Grid Duel example (Chaser vs. Runner on a 2D grid): what would each agent's fitness depend on, and how would the populations co-evolve?

10 Multi-Objective Evolutionary Algorithms

Multi-Objective Optimization Problems (MOOPs) involve several conflicting objectives (e.g. profit vs. weight), so there is generally no single optimal solution — only a set of trade-offs. Pareto dominance formalizes “better in every objective, strictly better in at least one,” giving rise to the Pareto-optimal set and front. SEMO is a simple baseline MOEA; NSGA-II improves on it with fast non-dominated sorting (fronts), crowding distance (diversity within a front), and elitism (combined parent+offspring selection).

10.1 Why is there generally “no single optimal solution” in multi-objective optimization, and what role do user preferences or constraints play in eventually picking one solution to deploy?

10.2 Give the formal definition of Pareto dominance for minimization: $x \preceq y$ if and only if ... (complete the two conditions).

10.3 Define the Pareto-optimal set P^* and the Pareto-optimal front PF^* .

10.4 Four candidate solutions have objective values (both minimized) $A = (2, 8)$, $B = (4, 6)$, $C = (6, 4)$, $D = (5, 7)$. Determine which solutions dominate which, and identify the non-dominated (Pareto-optimal) subset.

10.5 Describe the LOTZ (Leading Ones, Trailing Zeros) benchmark: what are its two conflicting objectives, and why is it useful for testing an MOEA’s diversity preservation along the Pareto front?

10.6 List and briefly explain the three core design issues for any MOEA: fitness assignment, diversity preservation, and elitism.

10.7 List NSGA-II’s three main innovations over SEMO and explain, for each, what problem of SEMO it addresses (slow convergence, poor diversity, scalability).

10.8 For the points sorted by objective F_1 (ascending): $A = 2, B = 4, C = 5, D = 7, F = 8$, explain why A and F (the boundary solutions) receive infinite crowding distance, and compute the crowding contribution (from F_1 alone) for point C using $\frac{f_{\text{next}} - f_{\text{prev}}}{f_{\text{max}} - f_{\text{min}}}$.

10.9 Explain why elitism in NSGA-II (combining parents and offspring before selecting survivors by front rank and crowding distance) helps prevent the loss of good non-dominated solutions across generations.

10.10 True or false, with justification: “The solution with the single best value in objective f_1 is always on the Pareto front.”

10.11 Describe the four possible shapes of a Pareto front depending on whether each of two objectives is minimized or maximized.

11 Evolutionary Machine Learning

Evolutionary Computation can address several ML challenges: feature selection (searching a 2^N -sized subset space), model structure/topology, model parameters (weights, replacing backpropagation), and hyperparameters. Direct weight encoding is simple but suffers from redundancy, costly crossover, and initialization sensitivity. Neuroevolution methods solve structure search more elegantly: NEAT co-evolves topology and weights with innovation-number-based crossover and speciation; CPPNs (evolved via NEAT, GP, GE, or DSGE) generate weight patterns as a function of node coordinates; DENSER evolves deep network topologies with a two-level GA+grammar (DSGE) representation.

11.1 List four ML challenges that Evolutionary Computation can help address (feature selection, model structure, model parameters, hyperparameters) and briefly state how EC contributes to each.

11.2 For feature selection over N features, explain why the search space has size 2^N , and how an EA represents a candidate feature subset.

11.3 When evolving the weights of a fixed-topology neural network directly (replacing backpropagation), what is the genotype, what is the phenotype, and what fitness function is typically used (in terms of the mean squared error E)?

11.4 Explain why a feedforward network's connection matrix only needs to encode the upper-right triangle, while a recurrent network's connection matrix needs the full matrix.

11.5 List three disadvantages of direct weight encoding (e.g. high redundancy, difficulty designing crossover, sensitivity to weight initialization, computational cost for large networks).

11.6 NEAT rests on three pillars: a novel encoding, speciation, and seeding with minimal-size networks. Explain why each pillar matters (what problem would arise without it?).

11.7 Describe NEAT's "add node" mutation: a connection is selected and disabled, and two new genes are inserted. Explain how innovation numbers, assigned when a connection is created, are later used during crossover to align genes from two different genomes.

11.8 In NEAT crossover, overlapping (matching) genes are inherited from either parent, while disjoint and excess genes are inherited from the more fit parent (assuming equal fitness is handled separately). Explain why this scheme allows NEAT to safely combine genomes of different sizes/topologies into a single child.

11.9 Given the compatibility distance $\delta = c_1 \frac{E}{N} + c_2 \frac{D}{N} + c_3 \bar{W}$, explain the role of each term and how δ is used, together with a threshold δ_t , to assign individuals to species.

11.10 Explain the biological motivation behind NEAT's fitness sharing formula $f'(i) = f(i) / \sum_j s(\delta(i, j))$, and what effect it has on a species that grows too large.

11.11 A Compositional Pattern Producing Function (CPPN) is a function $F(x_1, y_1, x_2, y_2) = w_n$ that outputs the weight between two nodes given their coordinates. How does this differ fundamentally from direct weight encoding, and why might it scale better to very large networks?

11.12 The “Using GP in NEAT” study compared CPPNs evolved by NEAT, GP, GE, and DSGE on two tasks (visual discrimination and line following). Summarize the empirical conclusion: was one approach uniformly best?

11.13 DENSER represents a deep network with two levels: a GA encoding the macro-structure (sequence of layers) and a DSGE (grammar-based) encoding of each layer’s parameters. Explain why this two-level design is useful compared to a single flat representation.

11.14 DENSER uses two crossover operators: one that swaps whole sets of layers between individuals, and one that swaps layers within the same non-terminal symbol/grammar category. Explain the difference and why both are needed.

11.15 True or false, with justification: “DENSER and other neuroevolution methods evolved networks that outperformed all human-designed architectures (VGG, ResNet, DenseNet) on every benchmark dataset tested.”

11.16 List DENSER’s mutation operators at the GA level (e.g. add/replicate/remove layer) and at the DSGE level (e.g. grammatical mutation, integer/float mutation).

Answer Key

1.1 (a) Learning (fitting parameters to data). (b) Design (constructing a structure satisfying requirements). (c) Optimization (minimizing an explicit objective $\|Dw - d\|^2$ over w).

1.2 Analytical solvers find an exact closed-form solution; numerical solvers find an approximate solution iteratively; observational solvers are data-driven. TSP has no known efficient analytical solver (NP-hard, no closed form for optimal tour as n grows), so a numerical/heuristic solver trades optimality guarantees for tractable runtime.

1.3 Representation (define the problem/search space), search strategy (explore potential solutions), quality measure (evaluate solution fitness). For knapsack: representation = binary inclusion vector; search strategy = e.g. local search or EA moves; quality measure = total value (zero if capacity exceeded).

1.4 False/overstated. Optimality guarantees are not the only criterion: heuristics are preferred whenever exact algorithms are intractable (exponential time) even if one technically exists, and often give “good enough” solutions far faster, which is the realistic trade-off in practice.

1.5 Both are optimization problems with no efficient closed-form/analytical solution and a vast or continuous search space, making them canonical illustrations for why heuristic/meta-heuristic search is needed regardless of the underlying domain (physics vs. combinatorics).

1.6 The reasoning is flawed. The NFL theorem averages over *all possible* fitness landscapes, including nonsensical, adversarially constructed ones. In practice, we only care about specific, structured problem classes (e.g. continuous multimodal functions, combinatorial routing problems), where algorithms that exploit problem structure provably and empirically outperform random search. NFL does not say “no algorithm is better than random on your problem”; it says no algorithm is universally best. Statistical tests comparing algorithms on a concrete benchmark class are entirely valid — NFL merely warns against claiming a result on one class automatically generalizes to all possible problems.

2.1 A meta-heuristic is a high-level, general-purpose algorithm for finding good (often approximate) solutions to complex optimization problems without guaranteeing optimality. Key characteristics: general-purpose (applies across domains), stochastic (randomized components), and balances exploration (broad search) vs. exploitation (refining good solutions).

2.2 Non-differentiable objective functions, non-convex search spaces (many local optima), high-dimensional / combinatorial complexity, and constraints producing irregular/disconnected search spaces.

2.3 Random Search samples independently with no feedback from past evaluations (no learning); Hill Climbing uses feedback by always moving toward improving neighbors. Random Search provides the baseline, since any useful algorithm should outperform pure random sampling.

2.4 B.

2.5 Local optima (gets stuck on the first peak found); plateaus (flat regions with no improving neighbor, so it stops prematurely); no exploration (purely exploitative, never escapes the local neighborhood once started).

2.6 High temperature makes $\exp(-\Delta E/T)$ close to 1 even for large ΔE , so worse moves are often accepted (exploration); low temperature makes the same probability near 0, so only improving moves are accepted (exploitation, like Hill Climbing). This probabilistic acceptance lets SA escape local optima that would trap Hill Climbing.

2.7 At $T = 1$: $P = e^{-2} \approx 0.135$. At $T = 10$: $P = e^{-0.2} \approx 0.819$. Higher temperature accepts worse moves far more readily.

2.8 $f(0, 0) = 20 - 10 - 10 = 0$ (the global minimum). $f(1, -1) = 20 + 1 + 1 - 10 \cos(2\pi) - 10 \cos(-2\pi) = 22 - 10 - 10 = 2$. Since $f(1, -1) = 2 > 0$, $(1, -1)$ is *not* the global optimum, yet it sits at the bottom of a small local dip surrounded by higher values in every immediate direction (Rastrigin has a regular lattice of local minima). Hill Climbing only ever looks at its immediate neighbors and moves downhill; once it reaches $(1, -1)$, every neighboring point evaluates worse, so the algorithm stops there, permanently stuck, with no mechanism to “look past” the surrounding hill toward the deeper global minimum at $(0, 0)$.

2.9 Linear cooling decreases T at a constant rate (simple, may cool too slowly or too quickly); exponential cooling decreases T multiplicatively (fast initial drop, common default); logarithmic cooling decreases T very slowly (theoretically guarantees convergence but is impractically slow). Faster schedules risk premature convergence; slower schedules cost more runtime for better quality.

2.10 True for the deterministic variants (Simple/Steepest-Ascent), since the same greedy rule from the same start always reaches the same local optimum; false for Stochastic Hill Climbing, which selects probabilistically among improving neighbors.

3.1 An EA is a stochastic, population-based optimization technique inspired by natural selection (mutation, recombination, survival of the fittest). Key characteristics: population-based, fitness-driven, stochastic, iterative.

3.2 EAs only need to evaluate a fitness function at sampled points (black-box), never requiring derivatives; gradient-based methods fail when the function is non-differentiable, noisy, or has many local optima, which EAs can escape via population diversity and stochastic variation.

3.3 Representation (binary inclusion vector for knapsack); Fitness evaluation (sum of values, 0 if over capacity); Selection (roulette wheel / tournament choosing parents by fitness); Variation (flip mutation, one-point crossover producing offspring).

3.4 Roulette Wheel: selection probability is directly proportional to fitness (a “slice” of the wheel). Tournament: a random subset competes, and the fittest of that subset wins, so selection pressure is tunable via tournament size k rather than tied directly to absolute fitness values.

3.5 Setting $\text{Fit}(x) = 0$ for any constraint violation, however small, treats a barely-infeasible solution the same as a maximally-infeasible one, discarding useful gradient information that a proportional penalty would preserve, and can make the search needlessly harsh near the feasibility boundary.

3.6 Elitism guarantees the best individual(s) survive unchanged to the next generation, preventing loss of the best-found solution. Risk: too large an elite size reduces effective population turnover/diversity, increasing the chance of premature convergence.

3.7 A. $P(B) = 20/50 = 0.4$; $P(\text{twice}) = 0.4^2 = 0.16$.

3.8 D.

3.9 Computational expense for large populations/generations $\rightarrow$ mitigate with smaller populations, surrogate fitness models, or parallelization. Sensitivity to parameter tuning (population size, mutation rate) $\rightarrow$ mitigate with self-adaptive parameters or systematic experimental tuning.

3.10 Largely true — fitness is the primary driver of selection — but not the whole picture: selection pressure is also shaped by the *mechanism* used (tournament size, elitism), and selecting *purely* on fitness with very high pressure risks premature convergence by discarding diversity that fitness alone does not capture. In practice, the selection scheme balances fitness-based pressure against maintaining a diverse population.

3.11 Generally true. Several algorithms in this course explicitly vary operator-related parameters over the run rather than keeping them fixed: CMA-ES adapts its mutation step size/covariance every generation, PSO’s inertia weight is decreased over time to shift from exploration to exploitation, and Simulated Annealing’s temperature follows a cooling schedule. A fixed rate cannot adapt to the changing needs of the search (more exploration early, more exploitation late), so allowing it to change is usually beneficial.

3.12 Yes. Since the wheel-slice probability $\text{Fit}(x)/\sum \text{Fit}$ is undefined or meaningless when fitness values are negative (it could yield negative or zero “probabilities”), one common fix is to shift all fitness values by a constant offset (e.g. subtract the minimum fitness in the population, optionally adding a small ϵ) so that every adjusted value is non-negative before computing the wheel proportions.

3.13 (a) **Full generational replacement** (as in (μ, λ) -ES): all parents are discarded and the next generation is drawn entirely from offspring. Advantage: maximum diversity turnover, prevents stagnation on any single parent. Drawback: no elitism — the best solution found so far can be lost if all offspring are worse. (b) **Steady-state replacement**: only one (or a few) individual(s) are replaced per step, keeping most of the population intact. Advantage: preserves accumulated quality; smooth convergence. Drawback: slow exploration; old individuals remain for many generations. (c) **Elitism**: the best k individuals always survive unchanged into the next generation (as in $(\mu+\lambda)$ -ES with $k = \mu$). Advantage: best solution

never regressed. Drawback: with large k , early good solutions monopolize the population and prevent exploration. (d) **Age-based replacement**: individuals are removed after a fixed number of generations regardless of fitness. Advantage: guarantees turnover; useful in dynamic environments where old solutions become stale. Drawback: good solutions can be discarded solely due to age, not inferior fitness.

4.1 Binary: discrete/combinatorial problems with natural 0/1 choices (e.g. knapsack). Real-valued: continuous numerical optimization (e.g. engineering parameter tuning). Permutation: ordering/sequencing problems (e.g. TSP, scheduling) where every value must appear exactly once.

4.2 One-point: swap all genes after a single cut point. Two-point: swap the segment between two cut points. Uniform: each gene independently swapped with some probability (e.g. 0.5). Uniform crossover disrupts building blocks (linked genes) the most, since it does not respect any locality/contiguity.

4.3 E.g. string 100100, flipping bit 5: 100110 — one (or more) bits are independently toggled with a small probability.

4.4 Uniform mutation replaces a gene with a value drawn uniformly from $[L_i, U_i]$ (large, unstructured jumps); Gaussian mutation adds noise $\sim \mathcal{N}(0, \sigma)$ centered on the current value, giving finer, more controllable step sizes (most changes small, occasional larger ones), which is generally preferred for local refinement.

4.5 $x' = 0.25(2) + 0.75(10) = 0.5 + 7.5 = 8.0$. α is the interpolation weight: $\alpha = 1$ gives x_1 , $\alpha = 0$ gives x_2 , and intermediate α gives a point on the segment between them.

4.6 It is “directional” because it always moves *from* the worse parent x_1 *toward and beyond* the better parent x_2 by a step proportional to α and the gap $(x_2 - x_1)$, actively exploiting the fitness difference; arithmetical crossover is purely an unweighted (or symmetric) blend with no notion of which parent is better.

4.7 Swap: exchange the positions of two genes. Insert: remove a gene and reinsert it elsewhere. Scramble: randomly shuffle the elements between two selected positions. A naive bit-flip cannot be used because permutations must remain bijections (each value exactly once); flipping one position would create a duplicate and a missing value.

4.8 The problem is producing a valid offspring permutation (no repeated or missing values) when recombining two permutations. Naive one-point crossover fails because splicing two permutation segments together typically duplicates some values and omits others; OX/PMX/Cycle Crossover each use different repair/mapping strategies to guarantee validity.

4.9 B (intended answer per the quiz). Caveat: strictly speaking, gray-coded binary already substantially mitigates Hamming cliffs, so this option is somewhat imprecise; by elimination among the four choices it is nonetheless the best available answer, since (a), (c), (d) are factually wrong.

4.10 D. $\Delta = |6 - 2| = 4$; range = $[2 - 0.5(4), 6 + 0.5(4)] = [0, 8]$.

4.11 (Self-check exercise — verify your repaired offspring contains each value exactly once.)

4.12 It cannot happen (probability 0). One-point crossover with a cut point $k \in \{1, \dots, n - 1\}$ produces offspring by exchanging the non-empty suffix after position k between the parents. Since every gene differs between the parents ($x_A[i] \neq x_B[i] \forall i$), the exchanged suffix is always different from the suffix it replaces, so the offspring can never be identical to either parent for any valid (non-trivial) cut point.

4.13 Representation: a length-81 vector of digits 1–9 (or one length-9 vector per row/block), with clue cells fixed and never mutated. Fitness (to minimize): total number of constraint violations — duplicate digits summed over all rows, columns, and 3×3 blocks. Crossover: exchange whole rows or whole 3×3 blocks between parents (preserves the easiest constraint to satisfy structurally if each block/row is kept as a permutation of 1–9). Mutation: swap two non-clue cells within the same block/row (keeps that block/row’s digit-multiset, and hence one constraint type, automatically valid).

4.14 False/overstated. A representation gap (indirect encoding, genotype $\neq$ phenotype) can be highly beneficial: it enables modularity and reuse (a single genotypic building block maps to a useful phenotypic component that appears across many structures), neutrality (multiple genotypes map to the same phenotype, buffering against destructive mutation — as in introns and GE wrapping), and richer/compressed search spaces (NEAT, CPPNs, DENSER all exploit this to avoid encoding millions of weights directly). The cost is added complexity in the mapping and potentially a harder-to-analyse fitness landscape. The

representation gap is beneficial when the problem has structure that the mapping can exploit; it is a drawback only when the mapping is arbitrary and adds noise without structure.

4.15 Representation: a binary vector of length n (one bit per Boolean variable; 1 = True, 0 = False). Every binary string is a valid assignment, so the entire search space is feasible — no repair is needed. Fitness (maximize): the number of satisfied clauses (ranges from 0 to m ; a perfect solution has fitness m). Variation: flip mutation (each bit independently flipped with probability p_m), one-point or uniform crossover. Selection: tournament or roulette wheel. Choice justification: GA with binary representation is the natural choice — the search space has 2^n candidate assignments and the binary string maps directly onto variable truth values. Unlike a constraint-satisfaction approach, the EA can make progress even when the formula is not yet fully satisfied, guided by partial clause counts.

5.1 Function set: e.g. $\{+, -, \times, \div, \sin\}$ (internal nodes). Terminal set: e.g. $\{x, 1.0, 2.0\}$ (leaves: variables and constants).

5.2 “Mutually exclusive” means each offspring is produced by *either* crossover *or* mutation, never both, chosen according to the given probabilities (e.g. 90%/10%). Subtree crossover swaps a randomly chosen subtree between two parent trees; subtree mutation replaces a randomly chosen subtree in one parent with a newly generated random subtree.

5.3 Full: every branch grows to the maximum depth (functions until the depth limit, then terminals). Grow: nodes are chosen from function *or* terminal set at every step, producing variable-depth, often smaller trees. Ramped Half-and-Half: 50% Full + 50% Grow across a range of depths, producing a population with varied shapes and sizes, which gives better initial diversity than either method alone.

5.4 D.

5.5 Introns occupy genetic material without affecting fitness, so when a destructive crossover or mutation event strikes an intron rather than functional code, the individual’s fitness is unaffected — introns act as a buffer/shield protecting the functional code from disruptive variation.

5.6 Bloat is excessive growth in program size without a corresponding fitness improvement. Causal theories: (1) introns (protect against destructive operators, so larger trees with more intron ‘padding’ survive); (2) search-space bias (there are more large programs than small ones at any given fitness level, so the search drifts toward larger trees); (3) crossover bias (crossover tends to produce many small, unfit offspring that are eliminated by selection, raising the average size of survivors). Countermeasures: penalize size in the fitness function; impose (dynamic) depth/size limits; use double tournament (first by fitness, then by size); use context-preserving crossover (choose the second cut point so the exchanged subtrees are of similar size).

5.7 GP is prone to overfitting because larger/more complex programs can essentially memorize the training fitness cases (related to bloat). Mitigations: prefer shorter/simpler programs (Occam’s razor) and evaluate on a separate validation set.

5.8 $(60 + 5)^2 = 65^2 = 4225$. Squaring makes the fitness function nonlinear: a combined error of 20 (units+distance) is penalised four times more than a combined error of 10, not twice. This disproportionate pressure on large errors creates a stronger gradient signal pulling GP programs toward solutions that get close to the cheese quickly, rather than treating all imperfect solutions roughly equally as a linear formula would.

5.9 The genotype is a variable-length string of integer codons. To expand a non-terminal with k production rules, the mapper computes (current codon value) mod k to select which rule to apply, consuming the codon and moving to the next non-terminal, until the derivation contains only terminals.

5.10 A. $26 \bmod 3 = 2 \Rightarrow$ index 2 = the 3rd rule.

5.11 Linear GP represents a program as a sequence of fixed-format instructions (op-code, argument registers, output register) operating on a shared register set; mutation probabilistically replaces one instruction element, and (two-point) crossover swaps a contiguous block of instructions between two parents.

5.12 B.

5.13 Function genes select which primitive (from a lookup table, e.g. $+, -, \times, \div$) a node computes; connection genes specify which earlier nodes/inputs feed into it. The levels-back constraint bounds how

far back a node may connect, guaranteeing the graph remains a valid, acyclic, feed-forward structure.

5.14 (Self-check exercise — node 2 computes $\text{input}_0 + \text{input}_1$ via function gene 0; node 3 computes $\text{input}_1 - \text{input}_0$ via function gene 1.)

5.15 Koza’s argument: crossover in GP exchanges entire subtrees, which he viewed as the primary source of novelty; the terminal and function sets are typically small and fully represented in the initial random population, so mutation adds little new genetic material. This reasoning has merit — in practice, crossover dominates (often used at 90%) and GP search works even with mutation rates near zero. However, the argument is not fully convincing: (1) when the population converges, crossover between similar individuals produces little variety, and mutation is the only operator that can reintroduce diversity; (2) empirically, a small mutation probability ($\approx 10\%$) consistently improves results in many domains; (3) subtree mutation also effectively implements random restart of a branch, which helps escape local optima. Modern GP systems use both; “no mutation needed” is a historically interesting position but not the consensus.

5.16 Standard division is undefined (or raises a runtime error) when the denominator is exactly zero. If a subtree can produce zero — which it can, since crossover and mutation can generate arbitrary sub-programs — then a tree containing a division node may become unevaluable, violating closure (the requirement that any function can accept any value from any other node as input). Protected division is defined as $\text{pdiv}(x, y) = x/y$ if $y \neq 0$, and a safe default value (commonly 1 or 0) if $y = 0$. This makes the function always produce a finite, well-defined output for any pair of inputs, restoring closure and ensuring every tree in the population is evaluable regardless of what values its subtrees compute.

5.17 The claim is largely accurate. **Redundancy:** many syntactically different trees compute identical functions (e.g. $(x + y)$ vs. $(y + x)$, or trees with inviable intron subtrees), so the genotype-to-phenotype map is many-to-one. This is not always bad — neutral variants (same phenotype, different genotype) act as a buffer against destructive variation (introns shield functional code) — but they also mean much search effort is spent on neutral moves that produce no fitness change. **Locality:** a representation has high locality if small genotypic changes produce small phenotypic changes. Tree GP has *low* locality because swapping even a small subtree can radically alter the program’s output over its entire input domain; two closely related trees can map to drastically different behaviors. Low locality makes the fitness landscape irregular and hard to navigate, which is one reason GP search is challenging and requires large populations.

6.1 ES are population-based optimization algorithms emphasizing Gaussian mutation and selection as the primary search drivers, with crossover playing a minor or absent role; GAs, by contrast, treat crossover (recombination) as the primary variation operator, with mutation as a secondary background operator.

6.2 A single parent generates one offspring via mutation; the better of the two (parent vs. offspring) survives to the next iteration. It is elitist because the current best is never discarded in favor of a worse mutant.

6.3 A.

6.4 Because (μ, λ) forces the next generation entirely from new offspring, it cannot get permanently stuck recycling the same parents, which encourages broader exploration of the search space at the cost of occasionally discarding a high-fitness parent.

6.5 A fixed mutation step size σ cannot simultaneously support effective broad exploration early in the search and fine-grained exploitation later (or adapt to a stretched/correlated fitness landscape).

6.6 Using the old mean $\mu^{(g)}$ (rather than the not-yet-finalized $\mu^{(g+1)}$) keeps the covariance estimate consistent with the population that was actually evaluated and selected at generation g , avoiding a circular dependency between computing the new mean and the new covariance from the same data simultaneously.

6.7 $\sigma_x^{2,(g+1)} = \frac{1}{2}[(4 - 5)^2 + (8 - 5)^2] = \frac{1}{2}(1 + 9) = 5$. A large σ^2 means the best individuals are spread far apart, so the next generation samples broadly around the new mean — exploration. As CMA-ES converges, the best individuals cluster near the mean and σ^2 shrinks, narrowing the sampling distribution and shifting to exploitation. The variance thus acts as an automatic exploration/exploitation indicator built into the algorithm’s state.

6.8 Simple ES and CMA-ES only use the best individuals each generation, discarding information from worse solutions about *where not to search*. NES instead optimizes the parameters (mean, covariance) of the full sampling distribution using the fitness-weighted gradient of *all* sampled points, retaining that

information.

6.9 The FIM accounts for the curvature/geometry of the distribution’s parameter space, so the natural gradient ($\text{FIM}^{-1}\nabla$) gives update steps that are invariant to reparameterization and better scaled than the raw Euclidean gradient, which can be misleading on a curved (e.g. non-Euclidean) parameter manifold.

6.10 Step 1: evaluate fitness of generation g . Step 2: isolate the N_{best} individuals. Step 3: compute $C^{(g+1)}$ from the N_{best} individuals and the old mean $\mu^{(g)}$. Step 4: sample the new generation from $\mathcal{N}(\mu^{(g+1)}, C^{(g+1)})$.

6.11 In simple ES, the mutation step size σ is a fixed algorithm parameter set before the run; it cannot respond to the changing needs of the search (broad exploration early, fine exploitation late) or to the geometry of the landscape (elongated valleys need anisotropic steps). Self-adaptation incorporates σ (or a full covariance matrix, as in CMA-ES) directly into each individual’s genome: both the solution parameters and the strategy parameters are mutated and selected together. Individuals whose σ is well-suited to the current landscape tend to produce better offspring and thus propagate their σ value. The result is that the algorithm automatically adjusts its mutation magnitude and direction over time without any externally prescribed schedule, which is more robust and typically gives significantly better performance on complex landscapes.

7.1 Initialization: generate a random population within bounds. Mutation: build a mutant vector from a weighted difference of solutions. Crossover: mix the mutant with the target vector (binomial crossover) to form a trial vector. Selection: keep whichever of target/trial has better fitness.

7.2 γ scales the difference vector, controlling the mutation step size. Very large γ causes overly disruptive, exploratory jumps (risking divergence); very small γ causes negligible variation, slowing convergence/exploration.

7.3 Without forcing at least one gene to come from the mutant vector, the binomial crossover could (with low probability) copy the target vector entirely, producing a trial identical to the target — wasting an evaluation with zero effective variation.

7.4 Deterministic: the trial replaces the target only if strictly better, with no probabilistic acceptance of a worse trial (unlike SA’s probabilistic acceptance of worse moves).

7.5 **best**: the base vector is the best individual found so far (rather than a random individual); **2**: two difference vectors are summed in the mutation; **bin**: binomial crossover. Using **best** increases selective pressure (faster convergence, less diversity); using 2 difference vectors increases diversity/exploration.

7.6 $v_i = [1.0, 2.0] + 0.5([4.0, 6.0] - [2.0, 3.0]) = [1.0, 2.0] + 0.5[2.0, 3.0] = [1.0, 2.0] + [1.0, 1.5] = [2.0, 3.5]$. $(x_2 - x_3)$ is the vector pointing from x_3 to x_2 — a direction in search space derived from two existing population members. Adding $\gamma(x_2 - x_3)$ to x_1 moves it in that same direction by a scaled amount. Unlike Gaussian mutation (which adds isotropic random noise with no memory of the population), DE’s perturbation is data-driven: its magnitude and direction adapt to the actual spread and structure of the current population.

7.7 The random schedule keeps a fairly constant average exploration level throughout the run (mild stochastic variation only); the linear schedule explicitly anneals from high γ_{max} (exploration) early to low γ_{min} (exploitation) late, mirroring SA’s temperature schedule.

8.1 A set of autonomous agents; no centralized control; an interaction topology/neighborhood together with simple rules of behavior.

8.2 Each Boid follows only simple local rules based on its nearby neighbors (no agent has global knowledge), yet the aggregate of these local interactions produces coherent, complex flock-level behavior (the collective pattern is not present in, nor predictable from, any single isolated individual).

8.3 Brave $\rightarrow$ inertial term (keeps moving in its current direction); Conservative $\rightarrow$ cognitive term (pulls back toward its own previous best position); Swarm $\rightarrow$ social term (pulls toward the swarm/neighborhood’s best position).

8.4 B.

8.5 D. $X_{t+1} = X_t + V_{t+1} = [4 - 2, -2 + 1.5] = [2.0, -0.5]$.

8.6 Clamping bounds each velocity component to $[-v_{\text{max}}, v_{\text{max}}]$ so particles cannot take excessively large

steps (avoiding large oscillations/divergence) while still allowing enough movement to explore; it is a simple mechanism to balance exploration (larger $v_{\max}$) against exploitation (smaller $v_{\max}$).

8.7 k_{inertial} starts high and linearly decreases toward a minimum across generations, so early iterations favor broad exploration (particles keep moving freely) and later iterations favor exploitation (particles settle and refine around good positions), mirroring SA’s cooling schedule.

8.8 The constriction coefficient χ multiplies the entire velocity update (inertial + cognitive + social) by a value < 1 derived analytically from $c_{\text{cognitive}} + c_{\text{social}}$, mathematically guaranteeing convergence/stability, whereas clamping is a simple, heuristic hard cap with no such guarantee.

8.9 sa measures the average distance particles moved between two consecutive generations; low sa signals the swarm has converged/lost diversity. Predator-Prey PSO adds a “predator” attracted to the global best and a repulsion term $R(d) = ae^{-bd}$ pushing prey particles away from it, injecting motion/diversity back into a converging swarm.

8.10 Each velocity component is passed through the sigmoid $f(v) = 1/(1 + e^{-v})$ to obtain a probability in $(0, 1)$; the corresponding bit is set to 1 if a random draw $r \sim U(0, 1)$ satisfies $r < f(v)$, and 0 otherwise — so larger positive velocities make a 1-bit more likely, larger negative velocities make a 0-bit more likely.

8.11 Stigmergy is communication mediated through the environment (chemical pheromone trails) rather than between individuals directly, so it is “indirect”: an ant senses a trace left by others rather than receiving a signal from another ant. This contrasts with direct physical (antennation, touch) and direct non-physical (stridulation, sound) communication.

8.12 Each ant starts at a random point and, at each step, stochastically chooses the next edge based on pheromone strength and heuristic information (e.g. inverse distance), until a full solution is built. At the end of the iteration, pheromone is evaporated on all paths and new pheromone is deposited on the paths used by ants, proportionally more for better solutions.

8.13 A. $\tau_A^\alpha \eta_A^\beta = 2 \cdot 9 = 18$; $\tau_B^\alpha \eta_B^\beta = 4 \cdot 1 = 4$; $p_A = 18/22 \approx 0.818$.

8.14 A.

8.15 $\alpha \gg \beta$: ants rely almost entirely on accumulated pheromone (history), risking premature convergence/stagnation on early-found paths. $\alpha \ll \beta$: ants behave almost like a greedy heuristic search, largely ignoring pheromone memory and behaving close to a memoryless constructive heuristic.

8.16 Standard AS: all ants deposit proportional to solution quality, which can dilute the signal from the best solution in a large colony. Elitist AS adds an extra deposit for the best-so-far solution, reinforcing it directly to speed convergence. Rank-Based AS instead weights every ant’s deposit by its rank, giving a smoother, less abrupt bias toward good solutions than pure elitism.

8.17 Each dimension of the particle’s continuous position encodes a “key” for one item in the permutation; sorting the items by their key values yields a valid permutation, letting standard continuous-space PSO operators be reused for ordering problems.

8.18 Random Immigrants periodically replaces a fraction of the swarm with freshly generated random particles, injecting new diversity and helping the swarm keep up with changing (dynamic) optima. The Forgetting mechanism resets or changes a particle’s remembered personal-best (local best) position, preventing the swarm from over-committing to a stale best-known point and encouraging renewed exploration.

9.1 Standard EA: $f : X \rightarrow E$, fitness depends only on the individual itself. Co-EA: $f : X_1 \times \dots \times X_n \rightarrow E$, fitness depends on a tuple of interacting individuals (possibly from different populations). This means individuals cannot be evaluated in isolation; fitness is inherently contextual/relational.

9.2 In competitive co-evolution, a solution is typically a single element of one X_i (e.g. one strategy that performs well against opponents), so the key questions are about whom to evolve against. In cooperative co-evolution, a solution is a combination of elements drawn from multiple X_i ’s working together, so the key question is how to encourage and reward cooperation.

9.3 Competitive: individual = a game-playing strategy, fitness = expected outcome against sampled opponents. Cooperative: individual = a single rule in a classifier system, fitness = overall accuracy of the classifier the rule contributes to.

9.4 Subjective fitness is computed using only the currently available elements (e.g. the current population of opponents); objective fitness is computed against a fixed, principled set (ideally all possible opponents). Subjective fitness is used in practice because objective fitness is usually computationally infeasible (the full opponent space is too large or unknown).

9.5 Separating candidate solutions S from tests T avoids overloading a single population with the dual, conflicting role of both searching for good solutions and providing the gradient/feedback for itself, which can otherwise lead to pathologies like cycling, since one population cannot simultaneously act as a stable, informative opponent and an improving solution-seeker.

9.6 Cycling: evolution rediscovers the same solutions repeatedly without net progress. Disengagement: opponents become either trivially easy or impossibly hard, giving no useful learning signal. Overspecialization (focusing): individuals master beating certain opponents while neglecting others, producing strategies that are brittle outside a narrow subset of the opponent space. Forgetting: strategies that defeated past opponents lose that ability once those opponents are no longer present to maintain selective pressure.

9.7 A.

9.8 Benefits: promotes modularity (clarity, smaller projected search spaces, individuals need not solve the whole task) and solution transparency. Central difficulty: credit assignment — determining how much of the joint solution’s success/fitness to attribute to each individual component, since individuals are rewarded for maximizing collaborations rather than directly for global optimality.

9.9 E.g. how to build a candidate solution/team from individual components; how to distinguish credit to the team vs. credit to individual team members; what learning bias best maintains diversity; the trade-off between specialization and generalization.

9.10 Round-robin: $\binom{10}{2} = \frac{10 \times 9}{2} = 45$ interactions. Single-elimination tournament: $n = 10$ interactions (one comparison per stage advancement, by the slide’s definition of fitness as “stage reached”).

9.11 Chaser’s fitness could reward minimizing time-to-catch the Runner; Runner’s fitness could reward maximizing items collected while maximizing survival time/distance from the Chaser. Both populations evolve simultaneously, each adapting to the other’s current strategies (a competitive, adversarial co-evolution).

10.1 Conflicting objectives mean improving one (e.g. comfort) typically worsens another (e.g. price), so no single point can be best in every objective simultaneously — only a set of non-dominated trade-offs exists. User preferences (or constraints, e.g. “weight < 2300”) are needed afterward to select one specific solution from this trade-off set according to what the decision-maker actually values.

10.2 $x \preceq y \iff (\forall i : x_i \leq y_i) \wedge (\exists k : x_k < y_k)$ — no objective of x is worse than y ’s, and at least one objective of x is strictly better.

10.3 $P^* = \{x \in S \mid \nexists x' \in S : f(x') \preceq f(x)\}$ (no other feasible solution dominates x). $PF^* = \{f(x) \mid x \in P^*\}$, the image of P^* in objective space.

10.4 $A = (2, 8)$ vs $B = (4, 6)$: neither dominates (A better in obj. 1, B better in obj. 2). $D = (5, 7)$ is dominated by $B = (4, 6)$ (B better or equal in both, strictly better in both). $C = (6, 4)$ is not dominated by any other (best in obj. 2). Non-dominated set: $\{A, B, C\}$.

10.5 f_1 = number of leading ones, f_2 = number of trailing zeros, in a binary string — these conflict because a 1 in a leading position cannot also count as part of a trailing run of zeros for the same bit. LOTZ is useful because its Pareto front is large and evenly structured, exposing whether an MOEA’s diversity-preservation mechanism actually spreads solutions across the whole front rather than clustering them.

10.6 Fitness assignment: how to convert multiple objectives into a selectable fitness signal (e.g. counting how many other individuals dominate a given one). Diversity preservation: ensuring the population spreads across the Pareto front rather than clustering (using a density metric in objective space). Elitism: preventing good non-dominated solutions from being lost across generations (e.g. via an archive).

10.7 Fast non-dominated sorting (assigns every individual to a front, giving a much more informative, graded fitness signal than simple dominance counting, addressing slow convergence); crowding distance (a density-based diversity metric within each front, addressing poor diversity/spread); elitism via combined parent+offsprings selection (addressing scalability/loss of good solutions in larger populations).

10.8 A and F are boundary solutions (smallest/largest F_1 value), and by convention are assigned infinite crowding distance to guarantee they are always preferred/retained, since there is no solution beyond them to compare a “next/previous” gap to. Contribution for C (using $A, B, C, D, F = 2, 4, 5, 7, 8$):

$$\frac{f_{\text{next}} - f_{\text{prev}}}{f_{\text{max}} - f_{\text{min}}} = \frac{7-4}{8-2} = \frac{3}{6} = 0.5.$$

10.9 By pooling parents and offspring before selecting survivors, a good non-dominated solution discovered in generation g cannot simply be discarded by the stochastic variation/selection of generation $g+1$, since it must still out-rank (by front and crowding distance) the newly created offspring to be replaced — non-dominated solutions are never lost purely by bad luck.

10.10 True. The solution with the single best f_1 value cannot be dominated, because domination of x requires y with $y_i \leq x_i \forall i$ and at least one strictly smaller; since no other solution can have an f_1 value smaller than the best f_1 , no solution can satisfy the “all objectives no worse” condition while being strictly better anywhere relevant to that extremal point. (It is, however, possibly tied with another point only if f_1 values are equal.)

10.11 min–min: front bows toward the origin (lower-left). min–max: front bows toward lower-right. max–max: front bows toward upper-right. max–min: front bows toward upper-left. In each case the front is oriented away from the combination of “best possible” values in both objectives simultaneously.

11.1 Feature selection (EC searches the combinatorial subset space); model structure/topology (EC searches discrete architectural choices, e.g. NEAT, DENSER); model parameters/weights (EC evolves weight vectors as an alternative to backpropagation); hyperparameters (EC searches/tunes settings like learning rate or layer sizes as genes).

11.2 Each of the N features can independently be included or excluded, giving 2^N possible subsets; an EA represents a candidate subset as a binary vector of length N (1 = feature included, 0 = excluded) and searches this space via standard GA operators.

11.3 Genotype: a real-valued vector of all the network’s weights. Phenotype: the neural network itself, with those weights plugged in. Fitness: $\text{fit} = \frac{1}{1+E}$, where $E = \frac{1}{N} \sum (y - \hat{y})^2$ is the mean squared error.

11.4 In a feedforward network, connections only go from earlier to later layers (no cycles), so only entries above the diagonal (later index > earlier index) can ever be non-zero, hence only the upper-right triangle is needed; a recurrent network allows connections in any direction (including backward/lateral), so the full matrix must be encoded.

11.5 High redundancy (many different weight vectors encode equivalent networks); hard to design an effective crossover operator (swapping weight sub-vectors between two networks usually destroys functional structure); strong sensitivity to weight initialization, requiring repeated runs and averaging to get a reliable fitness estimate; high computational cost for large networks with millions of weights.

11.6 Novel encoding: tracks structure (nodes/connections) explicitly, not just a fixed-length weight vector, allowing topology to evolve; without it, structural innovation (new connections/nodes) could not be represented or grow. Speciation: protects newly introduced structural innovations (which often start with lower fitness before being optimized) from being immediately eliminated by competition against already-optimized topologies; without it, innovation would be selected against too early. Minimal-size seeding: starts from the simplest possible topology and grows complexity only when useful, avoiding unnecessary bloat; without it, initial random complex topologies would waste search effort.

11.7 The mutation selects an existing connection, disables it, and inserts two new connection genes (input→new node, new node→output) plus the new node gene, each connection receiving a fresh innovation number. Because the innovation number is permanently tied to the historical origin of a gene, two genomes can be aligned gene-by-gene by matching innovation numbers during crossover, correctly identifying which genes are homologous (matching) vs. disjoint/excess, even though the genomes may have very different topologies.

11.8 Because innovation numbers provide a consistent historical alignment, matching genes (same innovation number in both parents) can be safely inherited from either parent without creating incompatible structure, while genes unique to one parent (disjoint/excess) are simply carried over from whichever parent is fitter — this avoids the need for any structural repair, since the offspring’s connectivity is always a valid subset/union of two already-valid topologies.

11.9 E/N and D/N measure the proportion of excess and disjoint (structurally different) genes relative

to genome size N ; $\bar{W}$ measures how different the matching genes' weights are on average; c_1, c_2, c_3 weight the relative importance of structural vs. weight differences. Two genomes are assigned to the same species if δ is below the threshold δ_t ; otherwise a new species is created.

11.10 Biologically, individuals in the same ecological niche compete for the same limited resources, so a large species should not be allowed to dominate the population purely by having many members. The formula divides an individual's raw fitness by the (density-weighted) number of same-species neighbors, effectively shrinking each species' total claim on selection in proportion to its size, which protects smaller/newer species and maintains overall diversity.

11.11 Direct encoding stores one parameter per individual weight (genotype size scales with network size); a CPPN instead stores a compact function that *generates* a weight from the coordinates of the two nodes it connects, so the genotype size is largely independent of network size — the same small CPPN can be queried to produce weights for arbitrarily large or differently-sized networks, scaling far better.

11.12 No single approach was uniformly best: in the visual discrimination task no approach was clearly superior to the others; in the line-following task GP outperformed all other approaches, and overall GP had a better average performance than NEAT across the study.

11.13 The GA macro-structure level controls high-level architectural decisions (which layers, in what order), while the DSGE level encodes the detailed parameters of each layer (filter sizes, activation functions, etc.) using a grammar tailored to that layer type. This separation lets crossover/mutation operate meaningfully at the right granularity (rearranging layers vs. tuning a layer's internals) and lets the framework generalize easily to new layer/network types by just changing the grammar.

11.14 The first crossover swaps entire sets of layers (a structural-level cut), exchanging whole functional blocks between parents; the second swaps individual layers within the same non-terminal/grammar category (a finer-grained exchange), letting two networks trade comparable layer types without disturbing the overall macro-structure. Both are needed to allow variation at both coarse (architecture) and fine (layer detail) granularity.

11.15 False. The benchmark results show DENSER sometimes *underperforming* the best human-designed network on a given dataset (e.g. on CIFAR-100, DENSER's accuracy was the lowest among the compared methods including ResNet, VGG, Fractional Max-Pooling, and DenseNet), so it is not true across every dataset tested.

11.16 GA level: add layer, replicate layer, remove layer. DSGE level: grammatical mutation (replace a grammar derivation choice), integer/float mutation (perturb a numeric layer parameter).